

Liste de liste (Nested Lists / Matrici)

Deoarece listele sunt extrem de flexibile, ele pot conține drept elemente alte liste. Aceste structuri se numesc liste imbricate (sau matrici) și reprezintă baza structurilor bidimensionale, asemănătoare unui tabel cu rânduri și coloane.

Sintaxa accesării: `matrice[index_rand][index_coloana]`

```
matrice = [
    [1, 2, 3], # Rândul 0 (Sertarul 0)
    [4, 5, 6], # Rândul 1 (Sertarul 1)
    [7, 8, 9] # Rândul 2 (Sertarul 2)
]

# Cum accesăm elementul "6"?
# Mai întâi alegem rândul (index 1), apoi elementul din rândul respectiv (index 2).
print(matrice[1][2]) # Va afișa 6
```

Exercițiul 1: Ai o tablă de joc simplificată (3x3), unde un pion reprezentat de litera "P" se află fix în centrul tablei. Spațiile goale sunt reprezentate printr-un spațiu " ".

```
tabla = [
    [" ", " ", " "],
    [" ", "P", " "],
    [" ", " ", " "]
]
```

Jucătorul mută pionul cu o pătrățică mai sus (înainte). Scrie codul care:

1. Șterge pionul de pe poziția veche (rândul 1, coloana 1), înlocuindu-l cu " ".
2. Plasează pionul "P" pe noua poziție (rândul 0, coloana 1).
3. Afișează starea finală a tablei de joc (rând cu rând, sau pur și simplu variabila tabla).

Exercițiul 2: Creează o matrice 3x3 (o listă care conține alte 3 liste) care să reprezinte o tablă de joc, plină de spații goale " ". Scrie un program care citește de la tastatură valoarea N (numărul de mutări). Pe următoarele N linii se vor citi 3 valori: j_id (1 pentru jucătorul X, 2 pentru jucătorul 0), rândul r și coloana c unde plasează simbolul. Plasează simbolul corect la poziția cerută și afișează matricea finală.

Pentru datele de intrare:

```
7
1 0 2 => jucătorul 1 plasează X la poziția 0, 2
2 1 1 => jucătorul 2 plasează 0 la poziția 1, 1
1 2 0 => jucătorul 1 plasează X la poziția 2, 0
2 2 2
1 0 0
2 0 1
1 1 0
```

Vom avea matricea finală:

X	0	X
X	0	
X		0